Module 2: Apprentissage Supervisé

Igor Colin, igor.colin@gmail.com

Telecom Evolution, Paris, France

Sommaire

1. Introduction
2. Classification binaire
3. Moyennes Locales
4. Ensemble Learning
5. Machines à Vecteurs Supports (SVM)
6. Réseaux de neurones
7. Références
8. Références

Sommaire

1. Introduction
2. Classification binaire
3. Moyennes Locales
4. Ensemble Learning
5. Machines à Vecteurs Supports (SVM)
6. Réseaux de neurones
7. Références
8. Références

Intervenants

- **Igor Colin**
  - Contact : `igor.colin@gmail.com`
  - Profil : Enseignement/Recherche/Conseil/Industrie
  - Web : http ://igorcolin.github.io/
  - Mots-clés : optimisation, apprentissage statistique, applications : vie privée, télécommunications, Internet des objets
- **Stephan Clémençon** (Telecom ParisTech - STA)
  - Contact : `stephan.clemencon@telecom-paristech.fr`
  - Profil : Enseignement/Recherche/Conseil/Industrie
  - Web : http ://www.tsi.enst.fr/∼clemenco/
- **Ons Jelassi** (Telecom ParisTech - Groupe STA)
  - Contact : `ons.jelassi@telecom-paristech.fr`
  - Profil : Enseignement/Recherche/Conseil

Cadre générique - apprentissage supervisé

- Couple de v.a. $= (X, Y) \sim P$ inconnue
- X = vecteur d'entrée à valeurs dans $\mathcal{X}(\mathbb{R}^d)$, ici $d \gg 1$
- Y = label/étiquette dans $\mathcal{Y} \subset \mathbb{R}$
- A priori, X modélise une information utile pour prédire Y
  Règle prédictive : $C : \mathcal{X} \to \mathcal{Y}$ choisie dans une classe $\mathcal{C}$
  (*e.g.* prédicteur linéaire $C(x) =^t \beta x + \alpha$)
- Fonction de perte : $\ell : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}^+$
- Risque (inconnu !) = Erreur de généralisation

$$L(C) = \mathbb{E}\left(\ell(Y, C(X))\right)$$

  à minimiser sur $C \in \mathcal{C}$.
- Données $= D_n = \{(X_1, Y_1), \ldots, (X_n, Y_n)\} \overset{i.i.d.}{\sim} P$

Sommaire

1. Introduction
2. Classification binaire
3. Moyennes Locales
4. Ensemble Learning
5. Machines à Vecteurs Supports (SVM)
6. Réseaux de neurones
7. Références
8. Références

Classification binaire

- Exemples : Prédiction de l'état d'un système (normal *vs* anormal), ciblage commercial, diagnostic médical, *etc.*
- $Y = \{-1, +1\}$
- Fonction de perte :

$$\ell(y, z) = \mathbb{1}_{\{y \neq z\}}$$

- Risque d'erreur :

$$\begin{aligned} L(C) &= \mathbb{P}\{Y \neq C(X)\} \\ &= \mathbb{P}\{Y \cdot C(X) < 0\} = \mathbb{E}\left(\mathbb{1}_{\{-Y \cdot C(X) > 0\}}\right) \end{aligned}$$

Approches Paramétriques

Régression logistique (1)

- Modélisation explicite de la probabilité *a posteriori* $\eta(x) = \mathbb{P}(Y = +1 \mid X = x)$
- On peut montrer qu'il est « naturel » de chercher $\eta(x)$ sous la forme :

$$\eta(x) = \frac{e^{f(x)}}{1 + e^{f(x)}}$$

- En prenant f linéaire, on obtient le modèle de la régression logistique (linéaire)

$$f(x) = \alpha + \langle \beta \cdot x \rangle$$

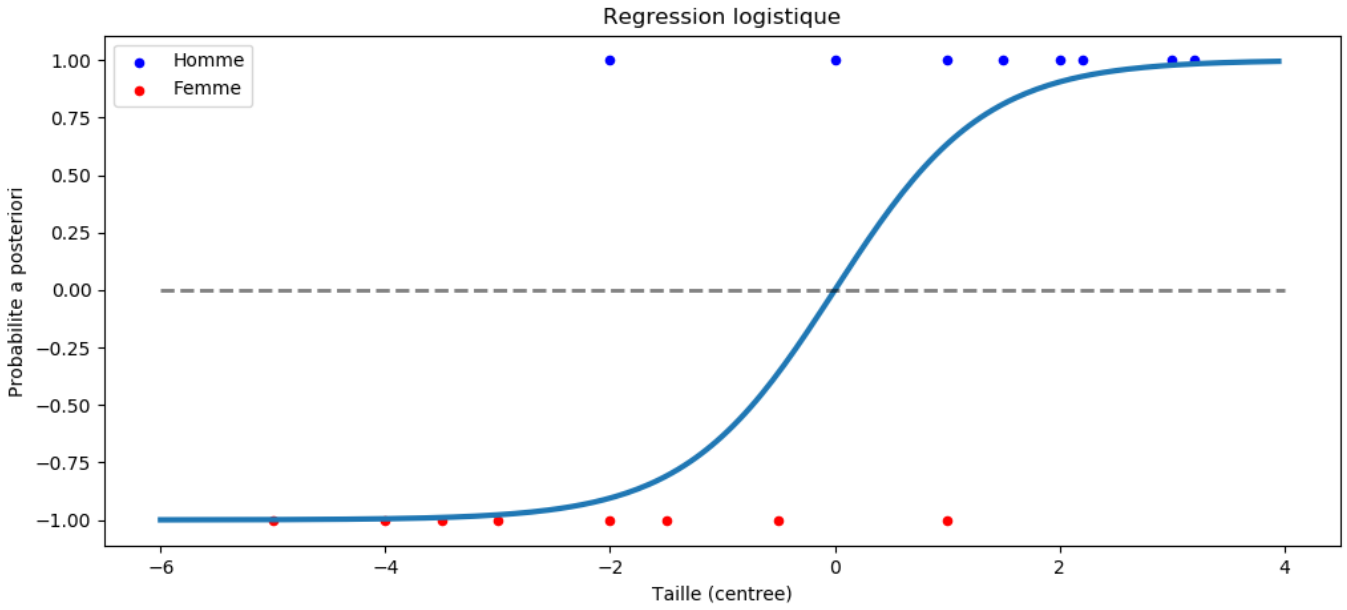
- Fonction de perte : inverse de la **vraisemblance**
- Minimisation du risque empirique

$$\min_{\alpha,\beta} - \sum_{i=1}^{n} \left\{ \frac{1 + y_i}{2} \log(\eta_{\alpha,\beta}(x_i)) + \frac{1 - y_i}{2} \log(1 - \eta_{\alpha,\beta}(x_i)) \right\}$$

Régression logistique (2)

- Comme pour *k-means*, il n'y pas de formule explicite pour le minimiseur du risque
- On utilise la descente de gradient (stochastique) pour obtenir les paramètres optimaux
- Avantages de la régression logistique :
  - Classifieur *plug-in* (prédiction rapide)
  - Donne une probabilité de label : indicateur de confiance
  - Se généralise bien
- Inconvénients :
  - Échoue si les données ne respectent pas le modèle linéaire
  - Parfois lente à entraîner avec la descente de gradient
  - Biais en grande dimension, même avec beaucoup de données

Logistique sur notre problème



Logistique vs. quadratique

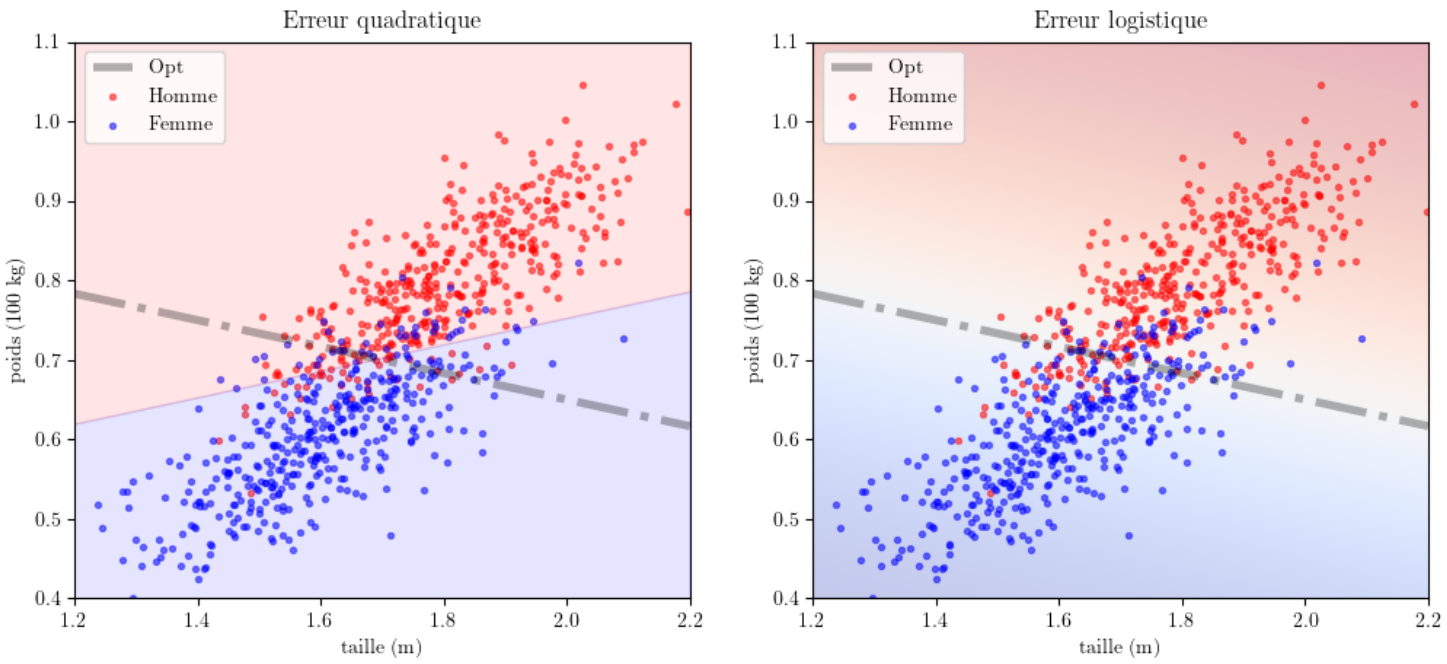


FIGURE – Comparaison des classifications sur notre problème.

Un mot sur la régularisation

- Des modèles tels que la régression logistique ont « peu » d'hyperparamètres
- Il est plus difficile de contrôler le surapprentissage
- C'est pourquoi il est courant de rajouter un paramètre « artificiel » contrôlant la complexité du modèle

Exemple classique : régression Ridge

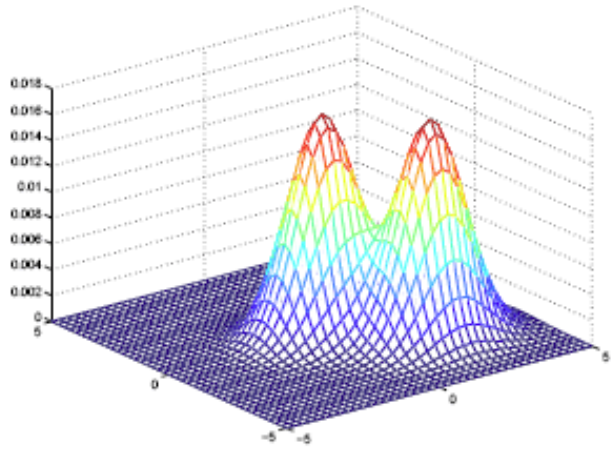
En régression linéaire ou régression logistique linéaire, il est courant de contrôler la complexité du modèle en imposant que le paramètre du modèle ne soit pas trop grand :

$$L(C_\beta) = \|\langle x, \beta\rangle - y\|^2 \quad \rightarrow \quad L_{\text{pen}}(C_\beta) = \|\langle x, \beta\rangle - y\|^2 + \lambda \cdot \frac{\|\beta\|^2}{2}$$

Le paramètre λ permet de contrôler le **compromis entre fidélité et complexité du modèle** : on le choisit par validation (croisée ou non).
Il existe énormément de pénalité possible, suivant les besoins du modèle ou la nature des données : $\|\cdot\|_1$, *elastic net*, *etc.*

Analyse Discriminante Linéaire

- **Hypothèse :** les lois conditionnelles de X sachant $Y = +1$ et sachant $Y = -1$ sont **gaussiennes** de même matrice de covariance Γ mais de moyennes distinctes μ_+ et μ_-
- C'est le cas de notre exemple jouet
- Analyse Discriminante Linéaire :
  - on estime empiriquement les moyennes et la covariance
  - on se sert des paramètres estimés pour prédire le label le plus probable



Analyse Discriminante Linéaire

Avantages :

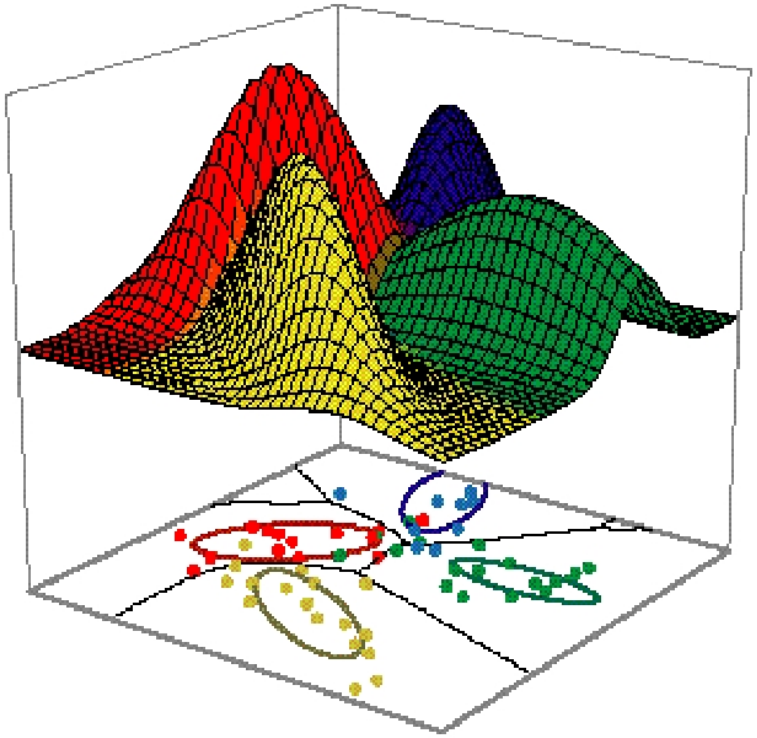
- Classifieur *plug-in* (prédiction rapide)
- Très simple et rapide à entraîner (3 calculs de moyennes)
- De bons résultats pour des données équilibrées

Inconvénients :

- Mauvais si données déséquilibrées
- Mauvais si les données ont des covariances radicalement différentes

Analyse Discriminante Linéaire - Extensions

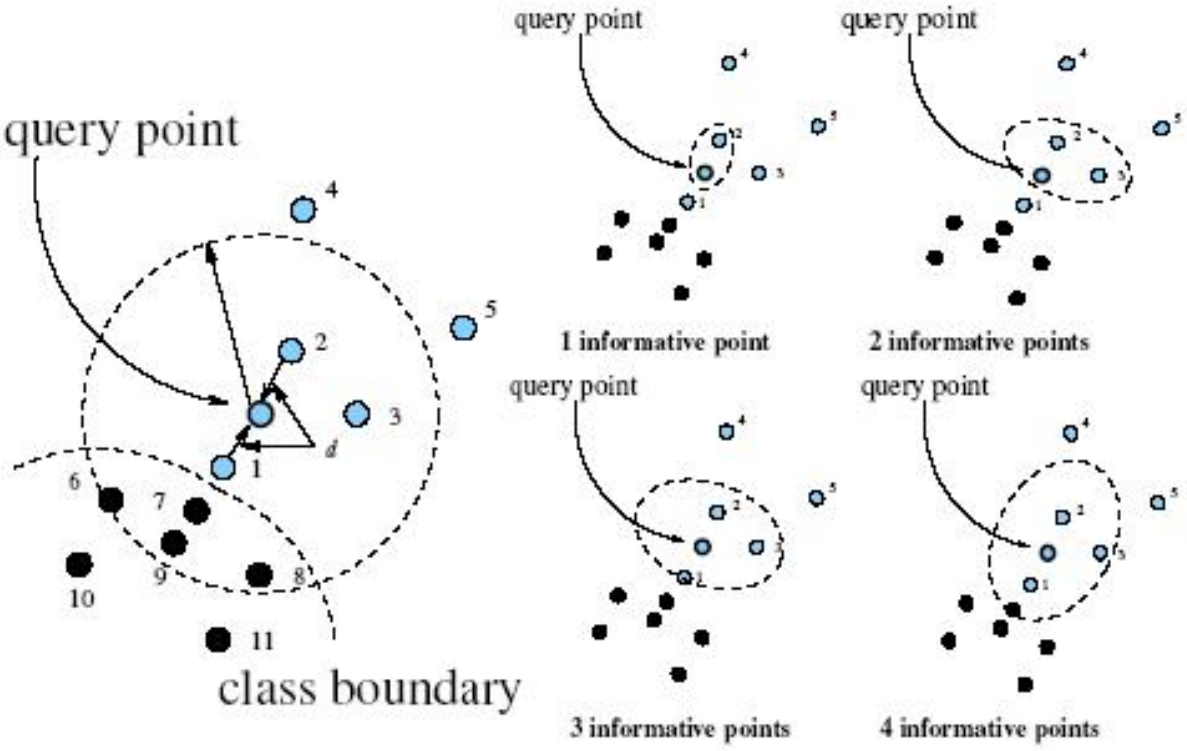
- **Naive Bayes** : on suppose que, sachant Y, les variables prédicitves $X^{(1)}, \ldots, X^{(d)}$ sont indépendantes
- Extensions **non linéaires** : analyse discriminante quadratique (QDA), mélange de Gaussiennes
- L'extension au **cadre multiclasse** est immédiate



Sommaire

1. Introduction
2. Classification binaire
3. Moyennes Locales
4. Ensemble Learning
5. Machines à Vecteurs Supports (SVM)
6. Réseaux de neurones
7. Références
8. Références

Une méthode nonparamétrique simple : les K-plus proches voisins



Les K-plus proches voisins

Consistance universelle (Stone '77)

Sous certaines conditions (simples) et si le nombre de voisins tend vers l'infini, le classifieur des K–plus proches voisins est consistant, c'est-à-dire

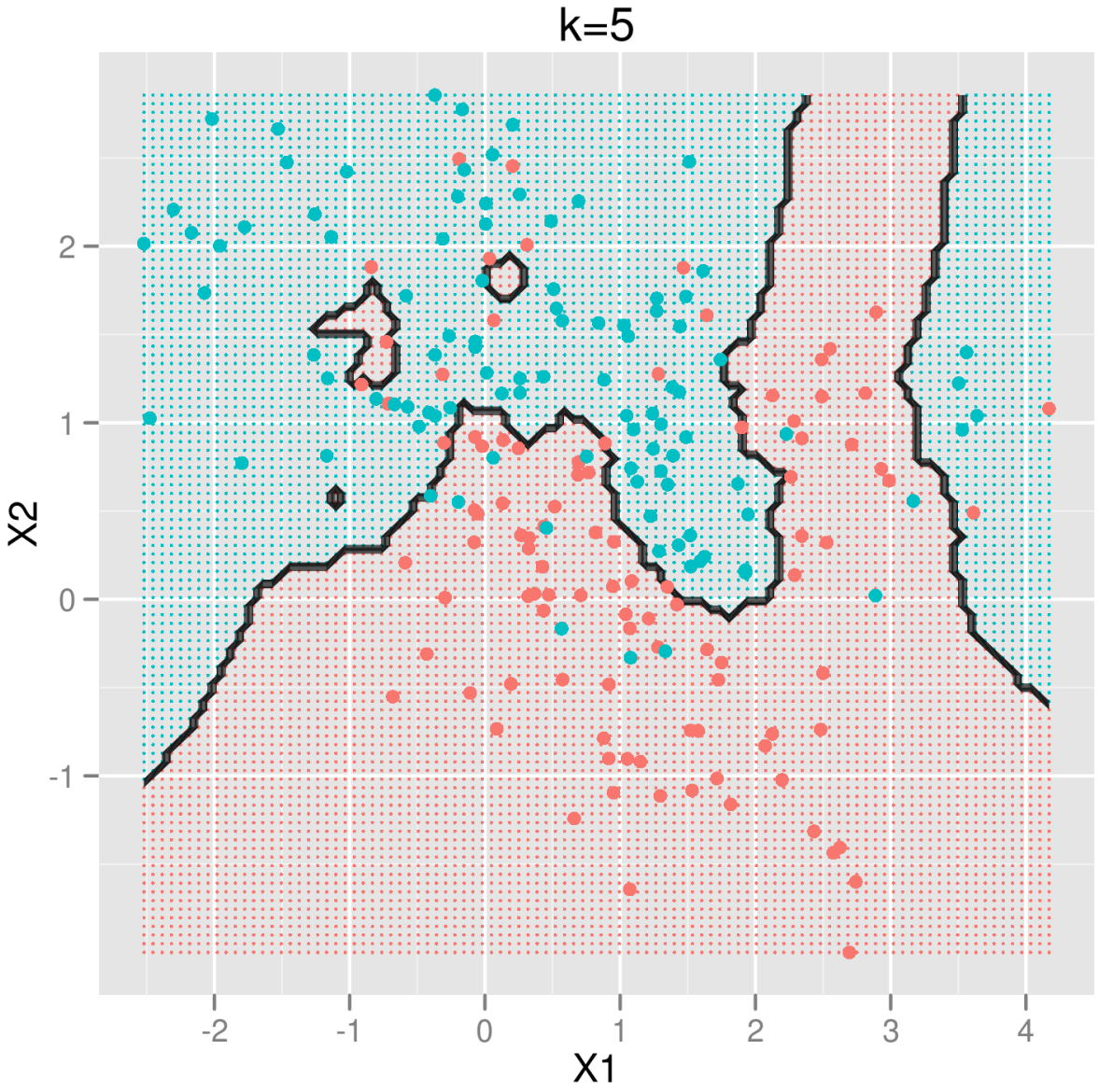
$$L(C_{K-NN}) - L^* \to 0, \quad \text{quand} \quad n \to \infty.$$

Autrement dit, son risque tend vers le risque du classifieur optimal.

Mais...

- La vitesse est **arbitrairement lente**
- Fléau de la dimension : ordonner les données est **coûteux en calcul**
- **Instabilité** : choix de K ? de la métrique D ?
- **Metric learning** (*e.g.* distance Mahalanobis distance)
- Variantes avec des **poids**

Les K-plus proches voisins
Une méthode trop flexible ?



Histogrammes - Moyennes Locales

- Les limites des K-plus proches voisins : le voisin le plus proche peut être très loin de X !
- Considérer une **quadrillage** de l'espace d'entrée :

$$C_1 \bigcup \cdots \bigcup C_K = \mathcal{X}$$

- Appliquer la **règle majoritaire** : si X tombe dans C_k, on prédit la classe présente en majorité dans C_k
- Cette règle revient à estimer la probabilité *a posteriori* au moyen d'histogrammes

$$\widehat{\eta}(x) = \sum_{k=1}^{K} \mathbb{1}_{\{x \in C_k\}} \frac{\sum_{i=1}^{n} \mathbb{1}_{\{Y_i = +1,\, X_i \in C_k\}}}{\sum_{i=1}^{n} \mathbb{1}_{\{X_i \in C_k\}}}.$$

Histogrammes - Moyennes Locales

- **Lisser** l'estimateur, la région de décision !
- Remplacer la fonction indicatrice par un **noyau de convolution**
- Idée : mettre en chaque point un ≪ pic ≫ orienté positivement ou négativement
- Exemples : noyau Gaussian, de Novikov, de Haar, *etc.*

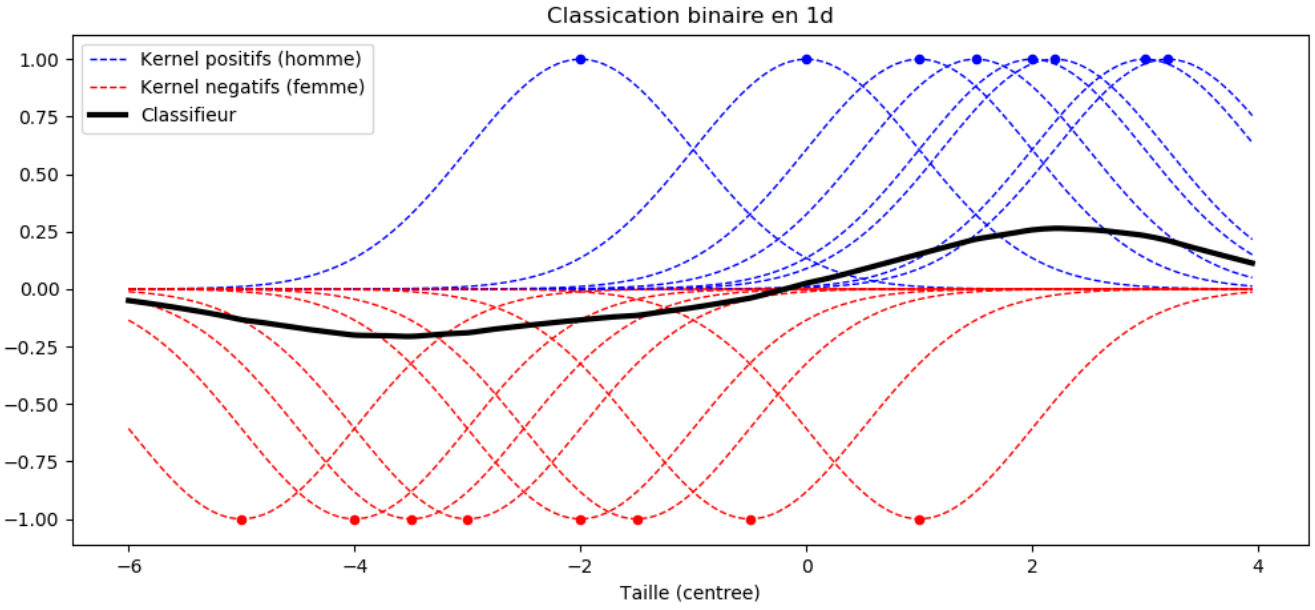


FIGURE – Estimateur par noyaux.

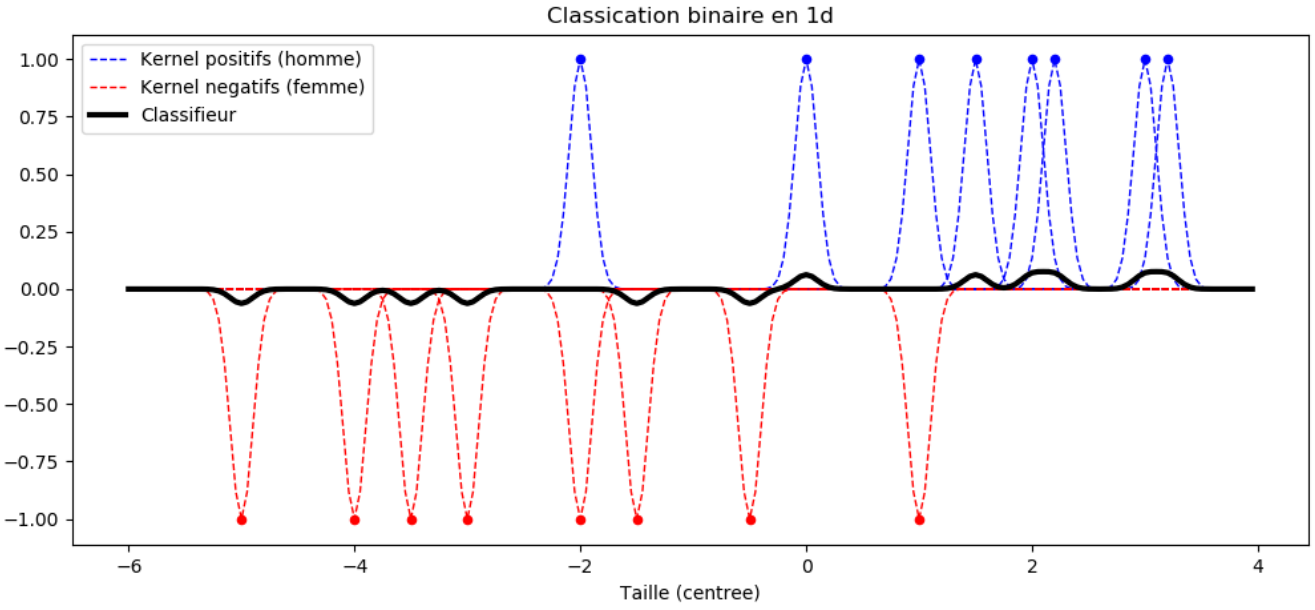


FIGURE – Estimateur par noyaux.

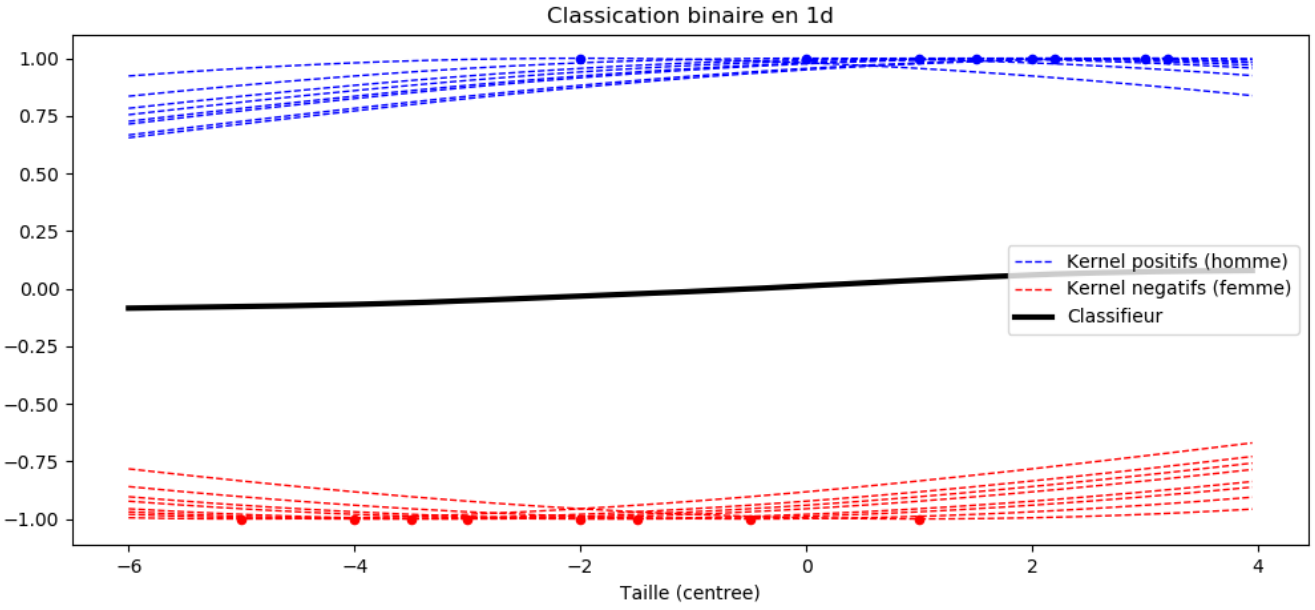


FIGURE – Estimateur par noyaux.

Méthodes à noyaux - Moyennes Locales

- En pratique, on observe de meilleurs résultats avec les noyaux que les histogrammes
- **Argument statistique :** Si η est une fonction "régulière", alors il vaut mieux l'estimer à l'aide de fonctions elles-mêmes régulières
- Gros problème de ces méthodes : dès que l'on augmente la dimension, de **très nombreuses** régions de l'espace sont incertaines :
  - aucun élément dans une case (histogramme)
  - trop éloigné de tout élément pour être significativement positif ou négatif (noyaux)

Arbres de classification : l'algorithme CART

- Pourrait-on choisir la partition **en fonction des données d'apprentissage** ?

Arbres de classification : l'algorithme CART

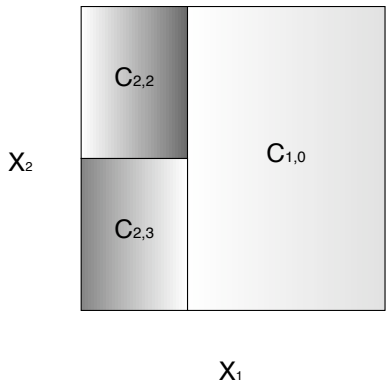
- Pourrait-on choisir la partition **en fonction des données d'apprentissage** ?
- The CART Book - Breiman, Friedman, Olshen & Stone (1986)
- Un algorithme de partitionnement récursif **glouton**

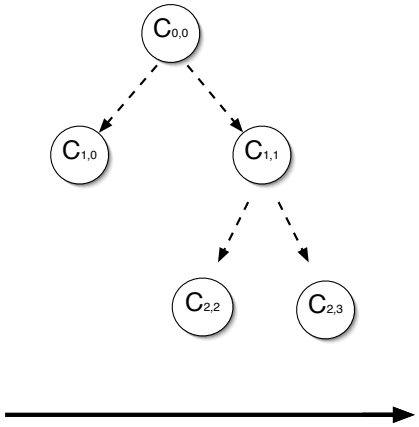
Arbres de classification : l'algorithme CART

- Données d'apprentissage $(X_1, Y_1), \ldots, ; (X_n, Y_n)$
- À toute étape de l'algorithme, le label associé à une région est le label majoritaire
- Initialement, notre seule région est l'espace tout entier
- Le classifieur initial est le classifieur constant, prédisant le label majoritaire
- À chaque étape, le but est de scinder une région en deux
- On choisit la scission diminuant le plus le risque empirique (algorithme glouton)
- On continue jusqu'à obtenir un nombre de région convenable où jusqu'à ce que l'on ne puisse plus améliorer le risque empirique

"Faire pousser l'arbre"

- La fonction de perte que l'on choisit est la perte binaire (ou 01)
- À chaque étape, on dit que l'on cherche la scission minimisant la mesure d'impureté
- En général, on se limite aux région orthogonales aux dimensions





Arbres de classification : l'algorithme CART

- Interprétabilité, visualisation
- Variables qualitatives
- Données incomplètes
- Quantification de l'importance relative des variables prédictives
- Randomisation
- Scissions diagonales
- Asymétrisation de l'erreur/impureté
- Extension au cadre multiclasse, à la régression
- Sélection de modèle : "meilleur sous-arbre, "élagage" rapide
- Algorithme alternatif : C4.5 (Ross Quinlan)

Arbres de classification : l'algorithme CART

- Interprétabilité, visualisation
- Variables qualitatives
- Données incomplètes
- Quantification de l'importance relative des variables prédictives
- Randomisation
- Scissions diagonales
- Asymétrisation de l'erreur/impureté
- Extension au cadre multiclasse, à la régression
- Sélection de modèle : "meilleur sous-arbre, "élagage" rapide
- Algorithme alternatif : C4.5 (Ross Quinlan)
- Mais... **performance prédictive moyenne** et **grande instabilité**

Sommaire

1. Introduction
2. Classification binaire
3. Moyennes Locales
4. Ensemble Learning
5. Machines à Vecteurs Supports (SVM)
6. Réseaux de neurones
7. Références
8. Références

Ensemble Learning
–
Bagging, Boosting et Forêts Aléatoires

En bref

- Ensemble Learning - Méthodes de Consensus
- Bagging - accroître la stabilité
- Boosting - "La meilleure technique sur l'étagère"
- "Le hasard fait bien les choses !" - les Forêts Aléatoires

Méthodes de Consensus

- Au lieu d'ajuster un unique classifieur, combiner les prédictions d'un **ensemble** de classifieurs

$$C_1(X), \ \ldots, \ C_M(X).$$

  Amit et Geman (1997)

- Vote majoritaire :

$$sign\left(\sum_{m=1}^{M} C_m(X)\right)$$

- Variante - vote majoritaire pondéré : $\alpha_i \geq 0$, $\sum_i \alpha_i = 1$

$$sign\left(\sum_{m=1}^{M} \alpha_m C_m(X)\right)$$

- Extension au cadre multiclasse, à la régression

Bagging - Agrégation Boostrap

- **B**ootstrap **agg**regat**ing** technique - Breiman (1996)
- Appliquable à tout algorithme $\mathcal{L}$
- A partir des données d'apprentissage $\mathcal{D}_n$:
  1. Générer indépendamment $B \geq 1$ échantillons bootstrap $\mathcal{D}_n^{*(b)}$ (par tirage avec remise dans $\mathcal{D}_n$)
  2. Pour b : 1 à B, mettre en oeuvre l'algorithme $\mathcal{L}$ à partir de $\mathcal{D}_n^{*(b)}$, produisant le classifieur $C^{*(b)}$
  3. Agréger les prédictions bootstrap en calculant le vote majoritaire :

$$C_{bag}(X) = sign\left(\sum_{b=1}^{B} C^{*(b)}(X)\right)$$

- Variante : si $C^{*(b)}(X) = sign(f^{*(b)}(X))$,

$$\widetilde{C}_{bag} = sign\left(\sum_{b=1}^{B} f^{*(b)}(X)\right)$$

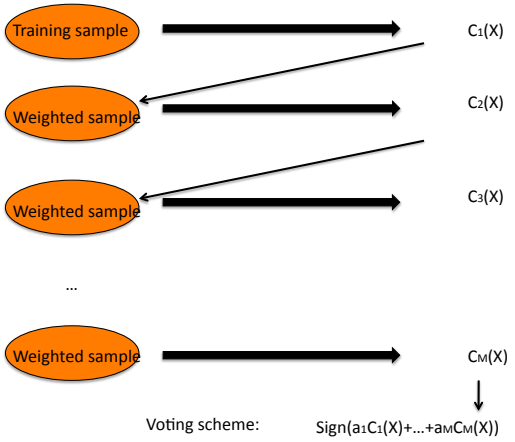
Bagging - Commentaires

- Le bagging peut **réduire significativement la variance** de procédures instables (ex : arbres de décision)
- La réduction de variance peut conduire à une erreur test moindre
- En classification :
  L'agrégation bootstrap d'un bon classifieur l'améliore,
  mais ...
  celle d'un mauvais classifieur peut le détériorer encore !

Boosting

- AdaBoost - Freund & Schapire (1995)
- L'ingrédient pour un "apprentissage lent", résistant au surapprentissage : une méthode de classification "faible" $\mathcal{L}$
- Heuristique :
  - appliquer $\mathcal{L}$ à des versions pondérées de l'échantillon original
  - accroître le poids des observations mal classées par la règle prédictive courante
  - agréger les classifieurs de façon non uniforme
    (un bon prédicteur ne devrait pas être construit à partir de quelques données aberrantes)
- AdaBoost surpasse ses concurrents sur la plupart des bases de données de référence
- Interpétation statistique : cinq ans plus tard...

Boosting - Schéma général



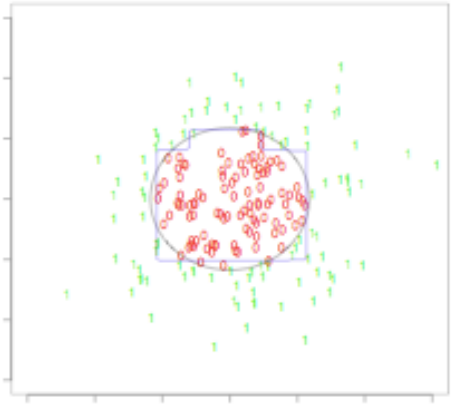
- Initialement, on donne la même importance à chaque observation dans le risque empirique
- À chaque étape, on augmente (resp. diminue) l'importance d'une observation si elle a été mal classée (resp. bien classée)
- Au final, on prédit grâce à un vote, en donnant plus d'importance aux meilleurs classifieurs

Aspects pratiques

- Comment mettre en oeuvre $\mathcal{L}$ à partir d'un échantillon **pondéré** ?
  - modifier le critère explicitement (ex : CART, SVM, k-NN, *etc.*)
  - tirer un échantillon d'apprentissage avec la distribution $\sum_i \omega_i \delta_{(X_i, Y_i)}$
- Quand faut-il stopper les intérations ?
  - tracer l'erreur test en fonction de M
  - on stoppe lorsque l'erreur test se stabilise

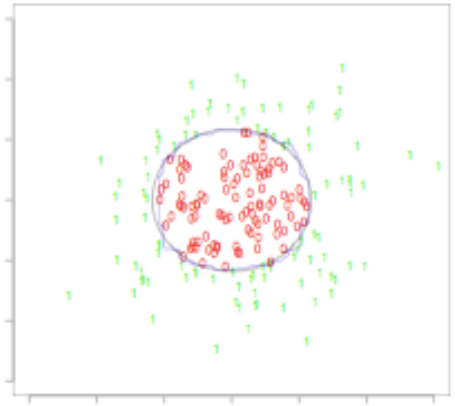
L'agrégation produit des régions "régulières"

Decision Boundary: Tree



When the nested spheres are in R^{10}, CART^{TM} produces a rather noisy and inaccurate rule $\hat{C}(X)$, with error rates around 40%.

Decision Boundary: Boosting



Bagging and Boosting average many trees, and produce smoother decision boundaries.

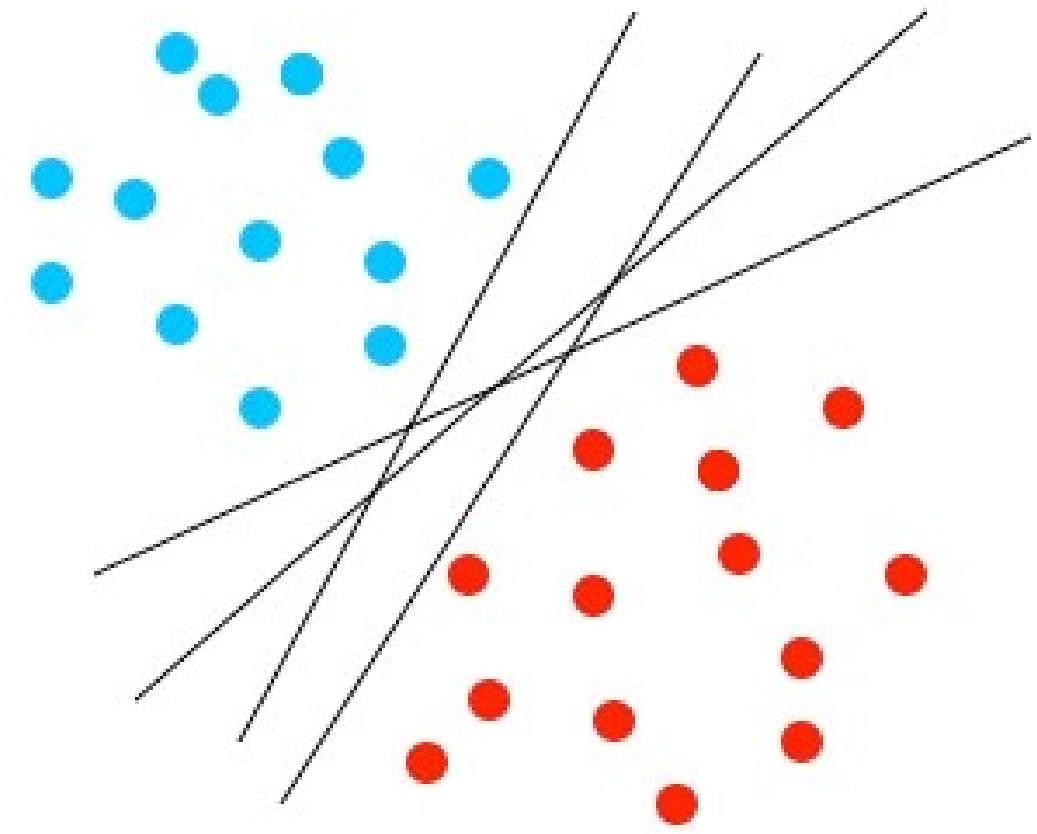
Forêts Aléatoires

- Ingrédients : bagging + randomisation
- Randomiser la collection de variables prédictives (*i.e.* les composantes de X) : avant de scinder chaque noeud d'un arbre de décision bootstrap
- Classifieur faible typique : arbre de faible profondeur, sans élagage
- L'agrégation préserve la consistance...
  mais aucune explication théorique de la performance observée !
- Heuristique : la randomisation "enrichit" la règle
- Randomisation des données d'apprentissage lorsqu'elles sont massives

Sommaire

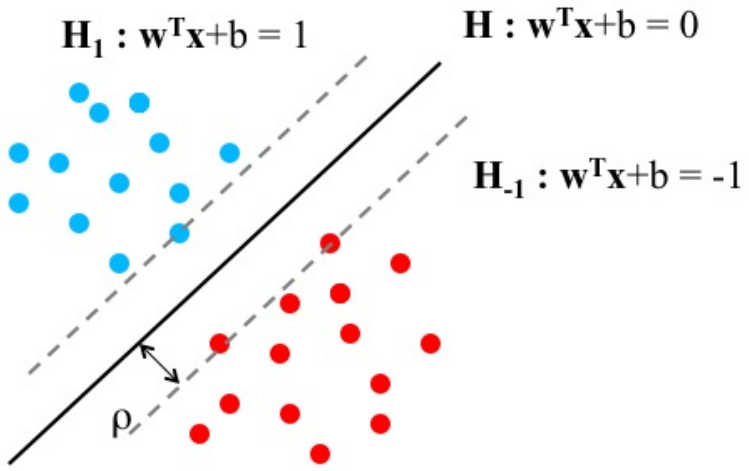
1. Introduction
2. Classification binaire
3. Moyennes Locales
4. Ensemble Learning
5. Machines à Vecteurs Supports (SVM)
6. Réseaux de neurones
7. Références
8. Références

Cas de données linéairement séparables



Motivation des SVM : quand des données sont séparables, quel classifieur choisir ?

Critère de marge



Idée : maximiser la **marge** entre les points et le classifieur

SVM linéaire : cas séparable

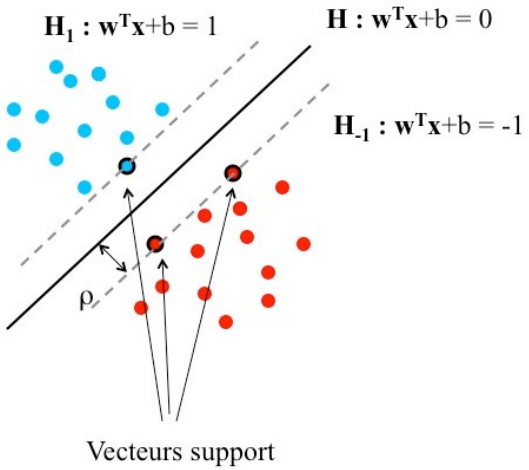
Ce problème se formule comme un problème d'optimisation

Optimisation dans l'espace primal

$$\begin{aligned} &\underset{\mathbf{w}}{\text{minimiser}} && \frac{1}{2}\|\mathbf{w}\|^2 \\ &\text{sous la contrainte} && y_i\langle \mathbf{w}, \mathbf{x}_i\rangle \geq 1,\ i = 1, \ldots, n. \end{aligned}$$

- $\mathbf{w}$ caractérise notre classifieur linéaire et $\langle \mathbf{w}, \mathbf{x}\rangle$ est sa prédiction pour le label de $\mathbf{x}$
- $\|\mathbf{w}\|$ est inversement proportionnel à la marge : on veut donc le minimiser
- $y_i\langle \mathbf{w}, \mathbf{x}_i\rangle \geq 1$ veut dire que chaque donnée est bien classée et du bon côté de la marge

Vecteurs "supports"



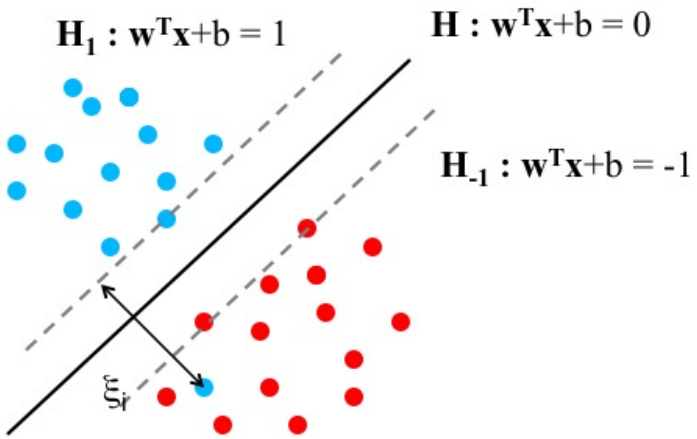
- Une étude minitieuse du problème d'optimisation montre que **w** est une combinaison linéaire des données !

$$\mathbf{w} = \alpha_1 \mathbf{x}_1 + \ldots + \alpha_n \mathbf{x}_n$$

- En réalité, c'est une combinaison linéaire des données **situées à la marge !**
- On appelle ces données, les **vecteurs supports**

Cas réaliste : SVM linéaire dans le cas données non séparables

Des données séparables sont peu réalistes (et peuvent être traîtées par des méthodes plus simples que les SVM)



Notion de marge douce

Cas réaliste : SVM linéaire dans le cas données non séparables

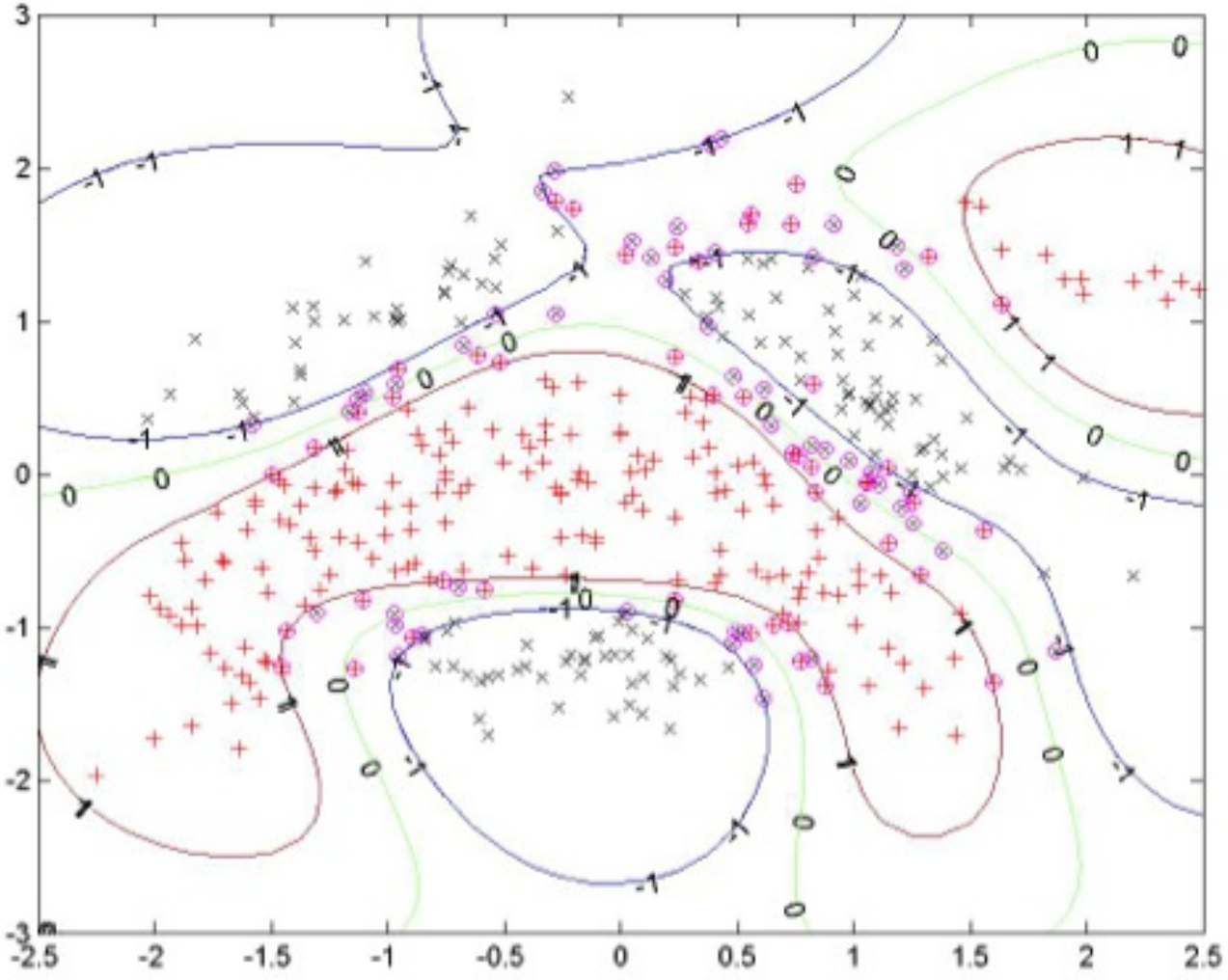
Introduire une variable d'écart ξ_i pour chaque donnée change notre problème d'optimisation :

SVM linéaires

$$\begin{aligned} \min_{\mathbf{w},b,\xi} \quad & \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i \\ \text{sous les contraintes} \quad & y_i(\mathbf{w}^T\mathbf{x}_i + \mathrm{b}) \geq 1 - \xi_i \quad i = 1,\ldots,n. \\ & \xi_i \geq 0 \quad i = 1,\ldots,n. \end{aligned}$$

- La variable ξ représente l'écart par rapport à la marge
- On inclut ces écarts dans l'objectif pour les limiter
- On a introduit un hyperparamètre : C
- Il encode le compromis entre une grande marge pour les points bien classés et peu d'erreur pour les points mal classés

Support Vector Machine : le cas non linéaire



Remarque

- Le problème de l'hyperplan de marge optimale ne fait intervenir les données d'apprentissage qu'à travers de produits scalaires (entre $\mathbf{w}$ et les données)

- Si je transforme les données « intelligemment », $\mathbf{x} \mapsto \varphi(\mathbf{x})$ et que je sais calculer facilement $\langle \varphi(\mathbf{w}), \varphi(\mathbf{x}) \rangle$, je peux peut-être faire un classifieur non-linéaire et plus adapté

- En pratique, on définit directement un **noyau**, $k(\mathbf{w}, \mathbf{x})$, équivalent à un produit scalaire

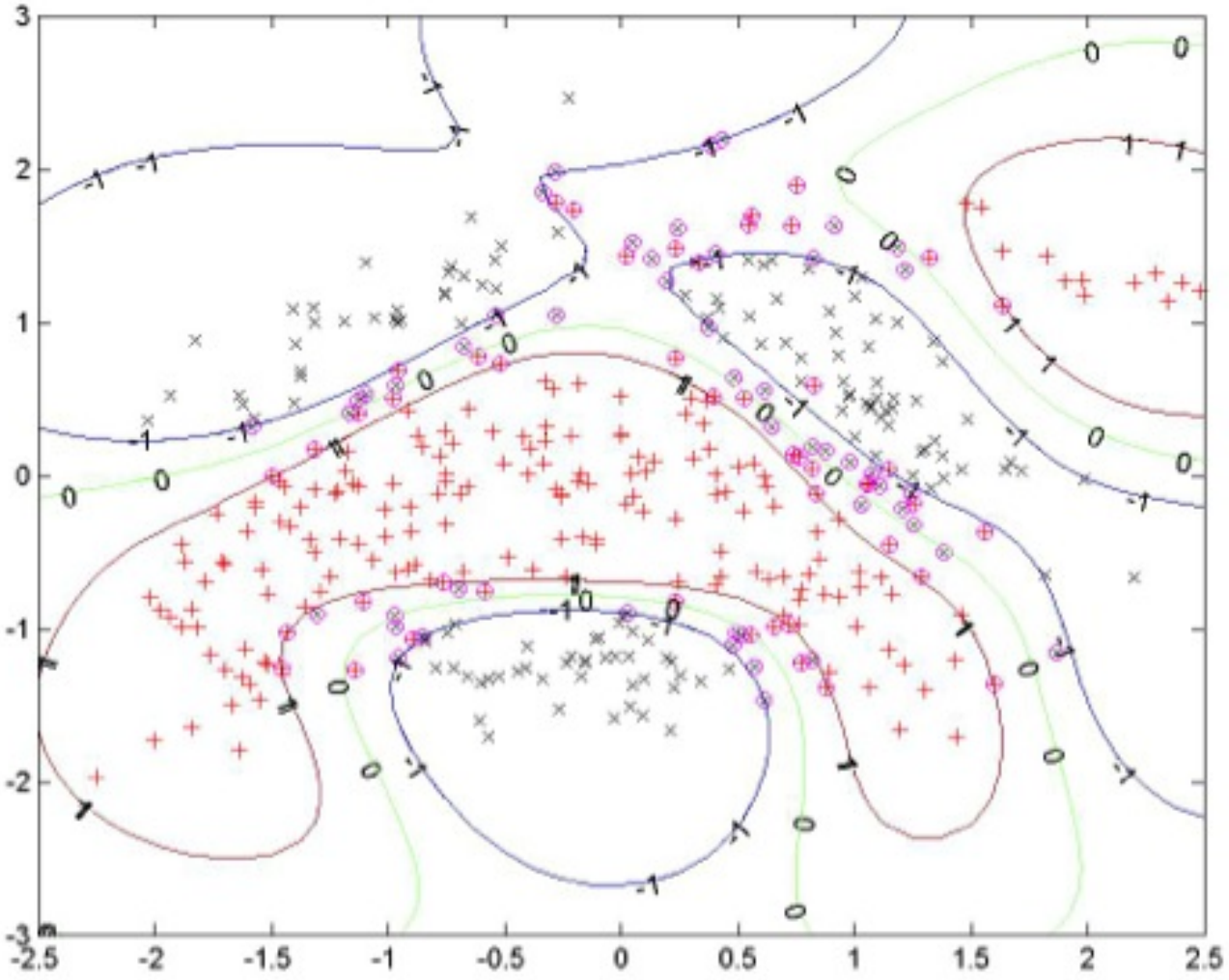
Noyaux

Noyaux typiques

$\forall \mathbf{x}, \mathbf{x}' \in \mathbb{R}^p$

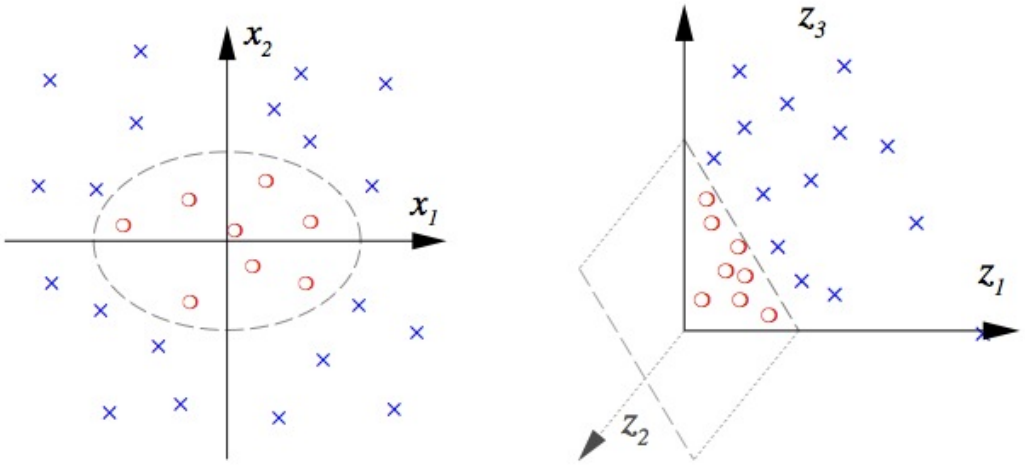
- Noyau linéaire : $k(\mathbf{x}, \mathbf{x}') = \mathbf{x}^T \mathbf{x}'$
- Noyau polynomial : $k(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^T \mathbf{x}' + c)^d$
- Noyau gaussien : $k(\mathbf{x}, \mathbf{x}') = \exp(-\gamma ||\mathbf{x} - \mathbf{x}'||^2)$

Support Vector Machine : séparateur non linéaire par noyau gaussien



Exemple : noyau polynomial

$$\Phi : \mathbb{R}^2 \to \mathbb{R}^3$$
$$(x_1, x_2) \mapsto (z_1, z_2, z_3) := (x_1^2, \sqrt{2}\, x_1 x_2, x_2^2)$$



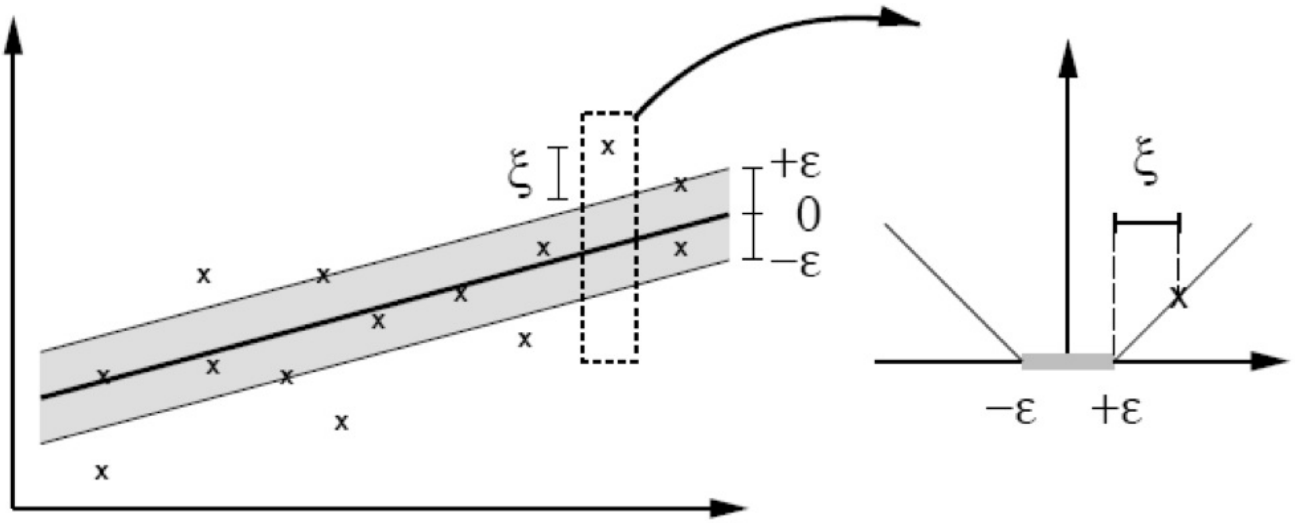
On remarque que $\varphi(\mathbf{x})^T \varphi(\mathbf{x}')$ peut se calculer sans travailler dans $\mathbb{R}^3$ (**astuce du noyau**)

Construction d'un noyau

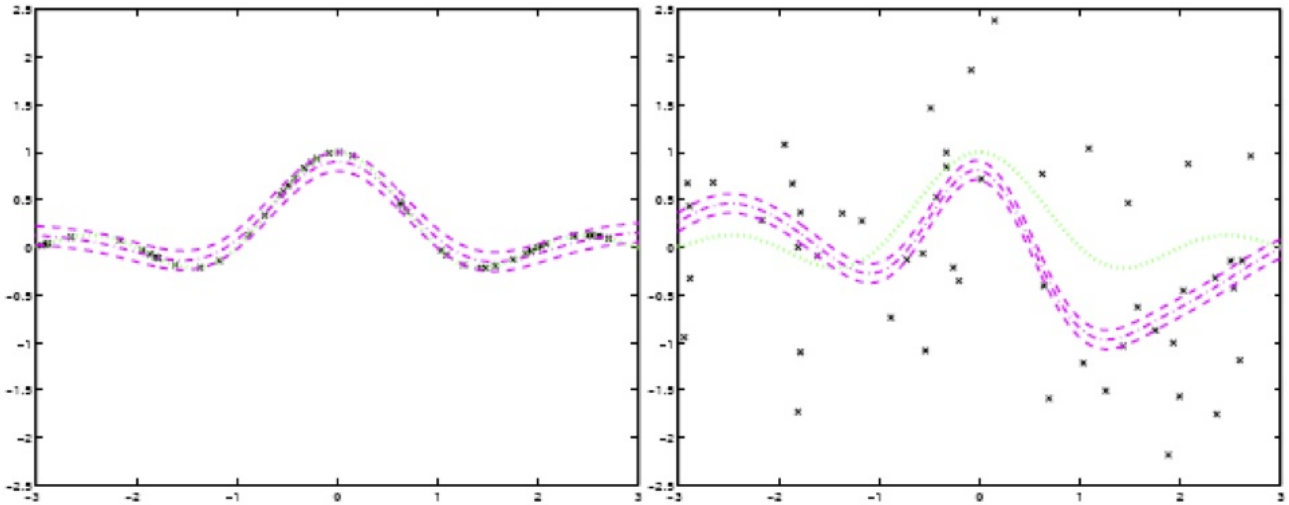
- Combiner des noyaux connus
- Des noyaux spécifiques à certains types de données :
  - **Objets structurés** : ensembles, graphes, arbres, séquences, ...
  - Données non structurées avec une structure sous-jacente : textes, images, documents, signaux, objets biologiques
- **Sélection d'un noyau** :
  - Hyperparameter learning : Chapelle et al. 2002
  - Multiple Kernel Learning : étant donnés $k_1, \ldots, k_m$, apprendre une combinaison convexe $\sum_i \beta_i k_i$ (see SimpleMKL Rakotomamonjy et al. 2008, unifying view in Kloft et al. 2010)

Support Vector Regression

- On peut étendre l'idée de marge à la régression
- Imposer un ε-tube : perte ε-insensible
- On reprend le même problème que précédemment en introduisant cette fois **deux marges**, puisque l'on souhaite limiter les dépassements des deux côtés cette fois



Support Vector Regression : example in 1D



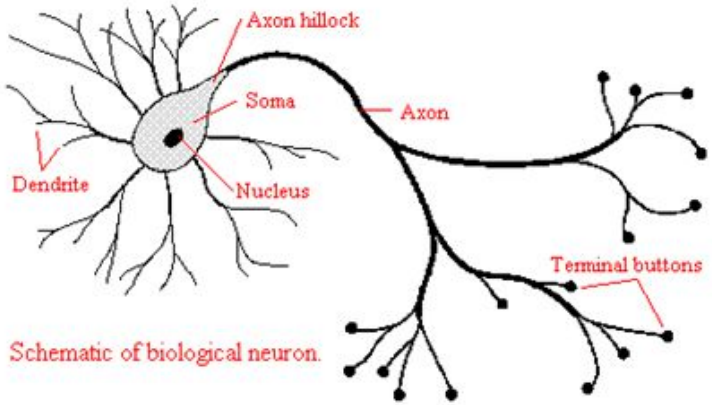
Identical machine parameters ($\varepsilon = 0.2$), but different amounts of noise in the data.

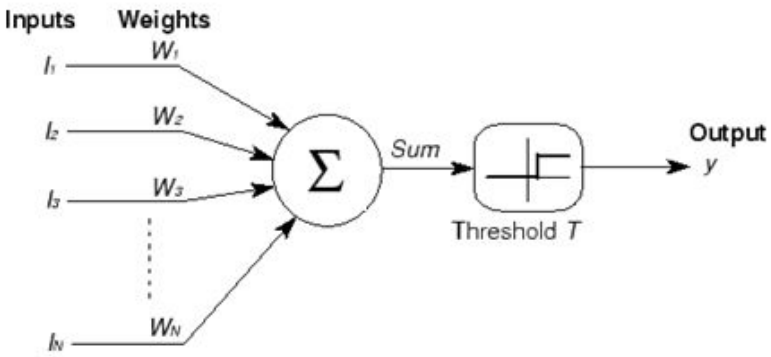
Sommaire

1. Introduction
2. Classification binaire
3. Moyennes Locales
4. Ensemble Learning
5. Machines à Vecteurs Supports (SVM)
6. Réseaux de neurones
7. Références
8. Références

Neurone

Le neurone

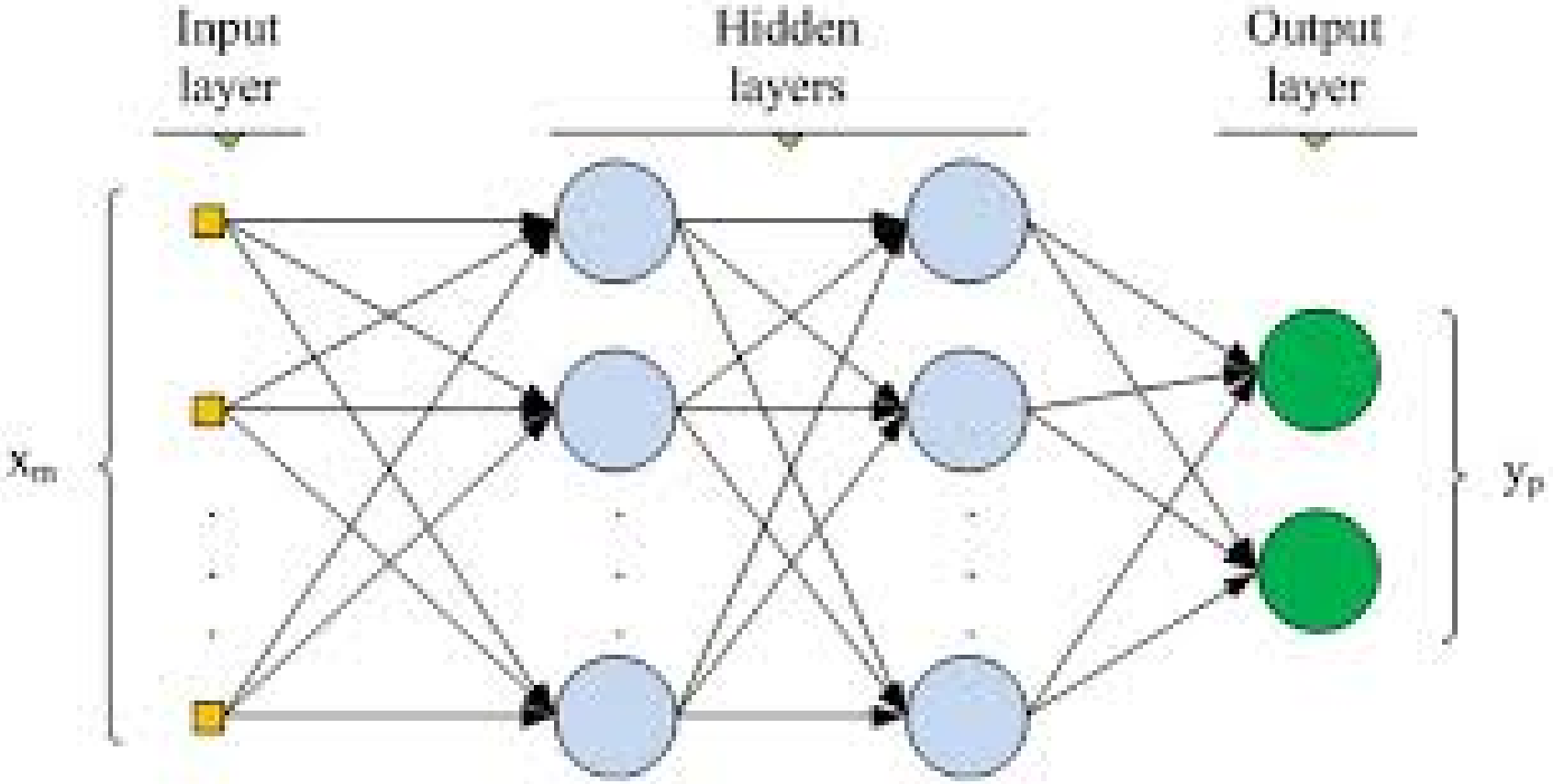




Neurone biologique

Neurone artificiel

Réseau de neurones formels (perceptron multi-couches)



Du neurone formel aux réseaux de neurones formels 1/2

- Neurone formel : Mc Cullogh et Pitts, 1943
- Règle d'apprentissage du perceptron, Rosenblatt, 1957
- Minsky et Papert : capacité limitée du perceptron, 1959
- Apprentissage d'un perceptron multi-couches par rétropropagation du gradient, Y. Le Cun, 1985, Hinton et Sejnowski, 1986.
- Perceptron multi-couches = approximateur universel, Hornik et al. 1991
- Convolutional networks, 1995, Y. Le Cun et Y. Bengio
- Entre 1995 et 2008, peu d'expansion du domaine (pbs fonctions non convexes, temps d'apprentissage, pas de théorie)

Du neurone formel aux réseaux de neurones formels 1/2

- Généralisation des GPU (processeurs graphiques) 2005
- Très large ensemble d'images : Imagenet, Fei-Fei et al. 2008 (maintenant 11 millions d'images)
- Réseaux de neurones de plus en plus profonds appris avec de gigantesques bases de données
- Apprentissage initial non supervisé (avec auto encodeur)
- Word2vec (Mikolov et al. 2013)
- Dropout (Srivastava et al. 2014)

Agenda

- Données non structurées
- Rappel : neurone formel ou perceptron
- Perceptron multi-couches
- Autoencodeurs
- Un mot sur les réseaux convolutionnels

Définition du neurone formel

Un neurone d'un réseau de neurons (profond ou non) est constitué :

- d'une fonction d'activation
- d'un vecteur de poids et un biais (intercept)

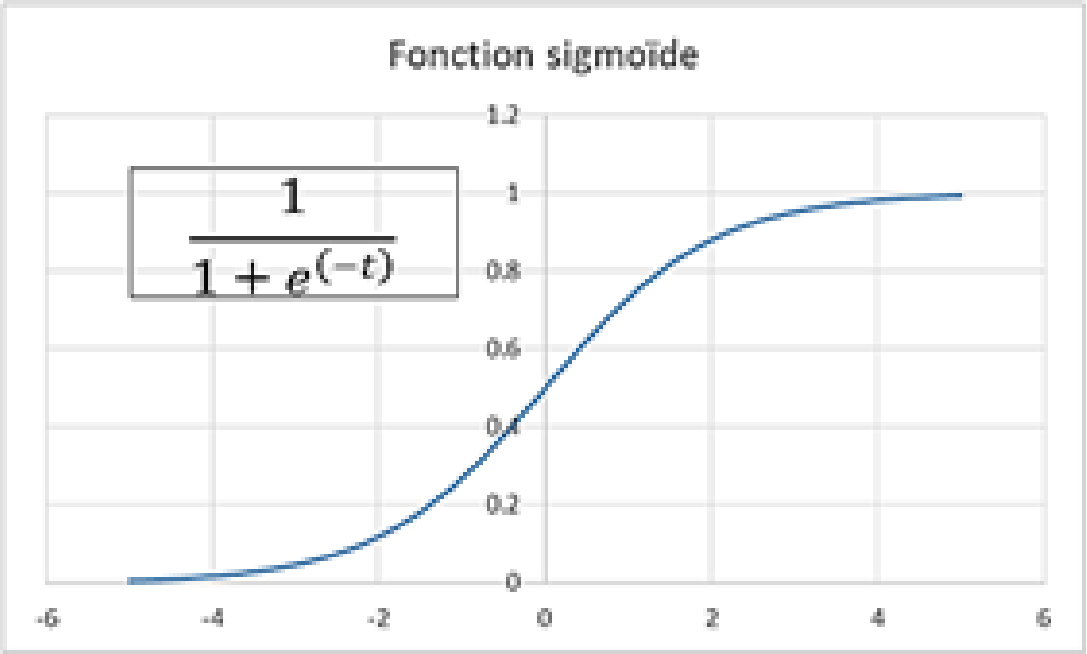
$$f(x) = g(w^T x + b)$$

La fonction d'activation a le même rôle que les fonctions en sortie des classifieurs (ex : sgn, sigmoïde, etc.). Des exemples typiques sont :

- ReLU : $x \mapsto \max(0, x)$
- La fonction *tanh*
- La sigmoïde

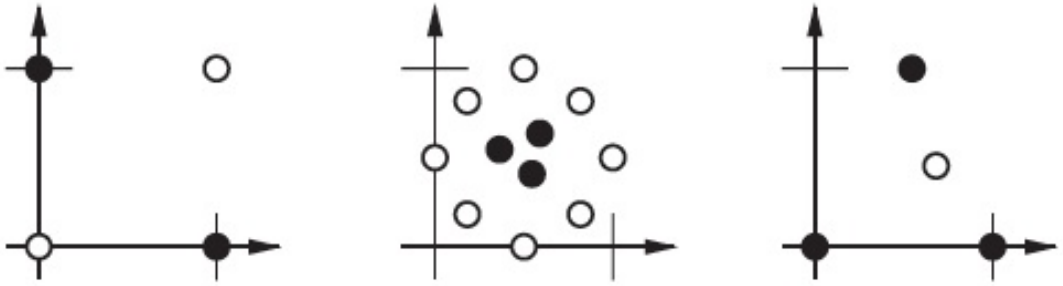
La sigmoïde et *tanh* sont simples à optimiser car différentiables (simplement) partout mais les ReLU ont souvent des bonnes propriétés empiriques

Fonctions d'activation du neurone formel



Limitation du neurone formel

Un unique neurone, tel quel, serait limité aux données linéairement séparables :



La force des réseaux de neurones vient des combinaisons possibles entre des dizaines, centaines voire milliers de neurones

Approximateur universel

En 1991, Hornik et al. démontrent que la famille des perceptrons à une couche cachée et à $p+1$ entrées est dense dans l'ensemble des fonctions continues de $\mathbb{R}^p$ dans $\mathbb{R}$. Un MLP à une couche cachée est un **approximateur universel**.

Approximateur universel

En 1991, Hornik et al. démontrent que la famille des perceptrons à une couche cachée et à $p+1$ entrées est dense dans l'ensemble des fonctions continues de $\mathbb{R}^p$ dans $\mathbb{R}$. Un MLP à une couche cachée est un **approximateur universel**.

D'autres exemples d'approximateurs universels que vous connaissez :

- Régresseur linéaire : NON
- SVM avec noyau universel tel que le noyau Gaussien : OUI
- Forêts aléatoires : OUI
- Boosting de stumps : OUI

Apprentissage à partir de données

Comme les autres méthodes étudiées jusqu'à présent, les réseaux de neurones rentrent (la plupart du temps) dans le cadre du risque empirique.

$$\mathcal{L}(W;\mathcal{S}) = \sum_{n=1}^{N} \ell(C(x_n), y_n)),$$

où W regroupe les paramètres de **chaque** neurone Étant données

les interactions complexes et entrecroisées entre les différents neurones, la fonction de risque empirique est extraordinairement **complexe**, difficilement interprétable et surtout **non convexe** : elle possède de nombreux minima locaux

- Le mieux que nous puissions faire, c'est trouver un bon minimum local
- C'est une des raisons que les MLP ont été pendant longtemps abandonnés en faveur des SVM/SVR plus faciles à optimiser (l'autre raison étant la puissance de calcul à disposition)

Algorithme d'optimisation

La rétropropagation du gradient

- Pour calculer le gradient, on utilise la règle de dérivation en chaîne $\frac{\partial L(W)}{\partial w_{ji}^{(1)}} = \frac{\partial L(W)}{\partial a_j} \frac{\partial a_j}{\partial w_{ji}^{(1)}}$
- Ainsi, on peut calculer le gradient en passant d'une couche à l'autre successivement
- Il est beaucoup plus efficace (en termes de mémoire) de commencer par la dernière couche et de remonter à la première : on parle de rétropropagation ou **backpropagation**
- Une fois toutes les couches parcourues, on met à jour les poids du réseau grâce à la méthode de descente de gradient (stochastique)
- Dans les bibliothèques de Deep Learning, on utilise parfois quelques variations de cette méthodes : une descente accélérée (Nesterov), Adam, etc.

La rétropropagation du gradient

Références :

Y. LeCun : Une procédure d'apprentissage pour réseau à seuil asymmétrique (a Learning Scheme for Asymmetric Threshold Networks), Proceedings of Cognitiva 85, 599-604, Paris, France, 1985.

Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986) Learning representations by back-propagating errors. Nature, 323, 533–536.

Régularisation et early stopping

Early Stopping

- Une première méthode de régularisation a été proposée dans les années 90 : il s'agit d'arrêter *a priori* l'apprentissage prématurément avant le sur-apprentissage
- On évite de se rapprocher trop près d'un minimum du risque empirique qui serait purement artificiel

Régularisation

- On utilise souvent une version **régularisée** du risque empirique :

$$\mathcal{L}(W) = \sum_n \ell(C(x_n), y_n) + \lambda ||W||^2$$

- On incite les poids du réseaux à ne pas être trop gros : des poids énormes sont souvent liés à du surapprentissage

Sélection de modèles

Le MLP a plusieurs hyperparamètres :

- Nombre de couches cachées
- Tailles des couches cachées
- Paramètre de régularisation λ
- Taille du minibatch
- Pas de la descente de gradient

La plupart sont trouvés par **validation croisée**.

Avantages et inconvénients des réseaux de neurones "feedforward"

Avantages

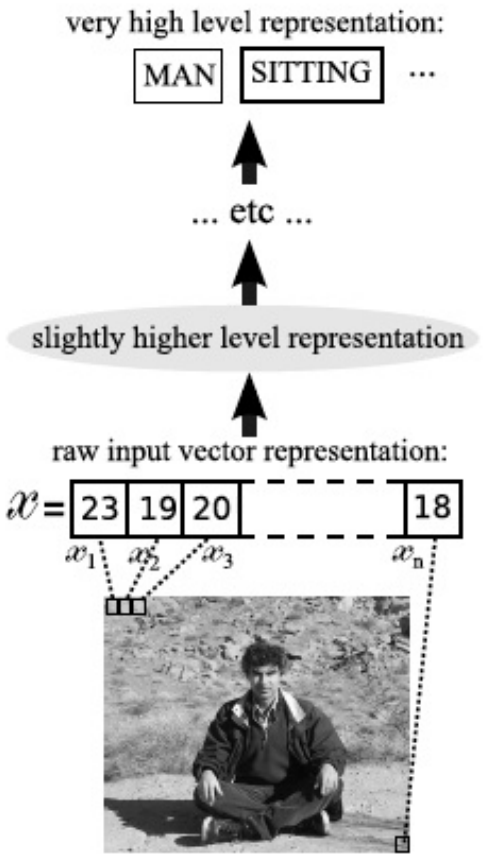
- Flexibilité au niveau des sorties : plusieurs classes, etc..
- Technique éprouvé depuis 1985
- Algorithme de gradient stochastique se plie bien aux besoins du **Big Data**
- Bénéficie des architectures GPU

Inconvénients

- Fonction de perte non convexe : pas de minimum global
- Descente de gradient nécessite souvent de nombreux ajustements
- Pas de cadre théorique
- Beaucoup de développements *ad hoc*

Réseaux dits profonds - Deep Learning

Image Y. Bengio



Apprentissage des réseaux dits profonds

A partir de 3 couches cachées, on parle de "deep learning", ce type de réseau est a priori intéréssant pour traiter des données complexes comme des images ou du texte.

Pourquoi utiliser plusieurs couches cachées ?

Même si un réseau à une couche cachée est en principe un approximateur universel, cela ne veut pas dire qu'un réseau à une couche cachée fournit la meilleure représentation et les meilleures performances.

Apprentissage des réseaux dits profonds

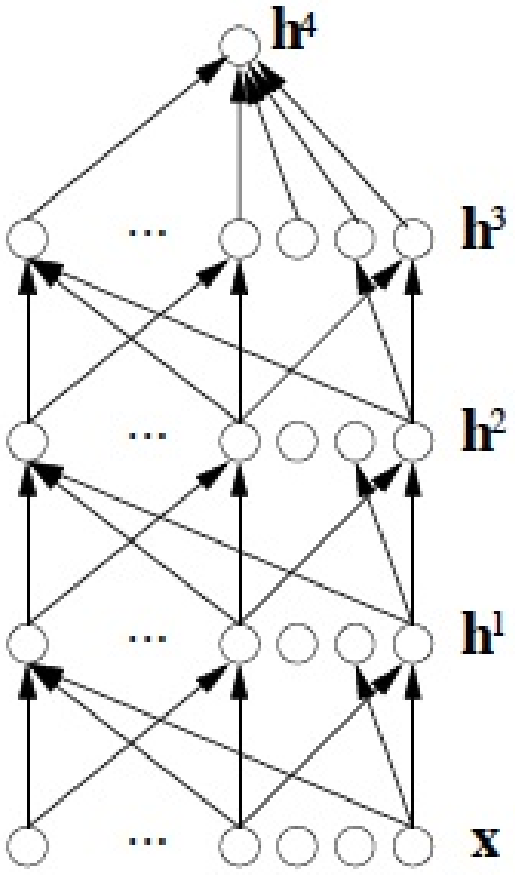
Malgré le risque de surapprentissage,

deux bonnes raisons de s'intéresser aux réseaux profonds

- l'amélioration des capacités de calcul et de mémoire (GPU)
- la disponibilité de gigantesques bases de données (Imagenet, Fei-Fei, 2008)

Apprendre un réseau profond par simple rétropropagation du gradient ne fonctionne pas si bien que cela (Bengio et al. 2007 ; Erhan et al. 2009).
Le réseau tombe dans des minima locaux très mauvais sans une bonne initialisation.

Apprentissage des réseaux dits profonds



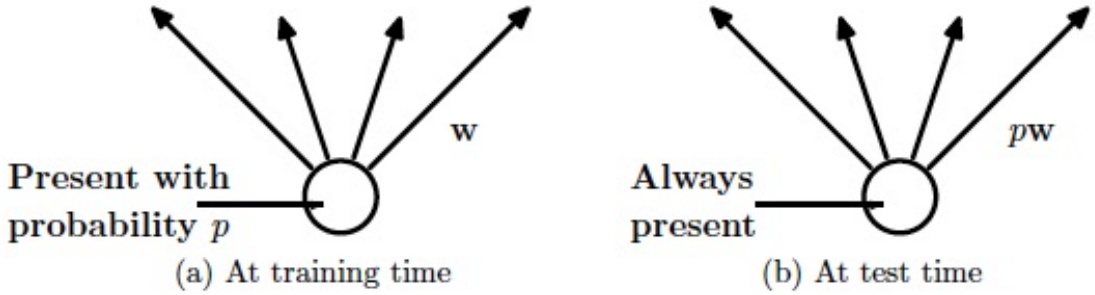
Apprentissage des réseaux dits profonds

Deux améliorations notables

- Dropout
- Auto-encodeurs

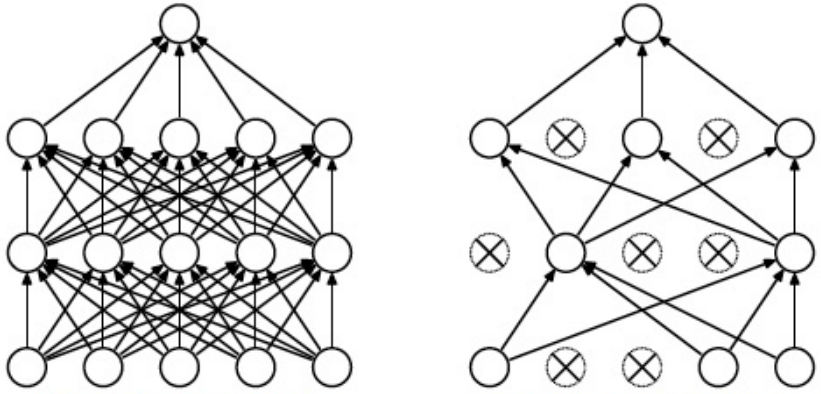
Eviter l'overfitting des réseaux profonds par *dropout 1/3*

Pour les réseaux profonds (>>2 couches) :



- Durant l'apprentissage, à chaque étape de gradient : chaque neurone est présent avec une probabilité p, ce qui veut dire que certains neurones ne sont pas présents et donc ne sont pas corrigés systématiquement.
- Pendant la prédiction (en test), chaque unité est présente et un facteur p est appliqué à ses poids.

Eviter l'overtfitting des réseaux profonds par *dropout 2/3*



(a) Standard Neural Net (b) After applying dropout.

Une interprétation :

Si on dispose de m neurones au total, c'est comme si on apprenait avec 2^m réseaux clairsemés et au moment du test, on superpose tous ces réseaux en un seul avec lequel on prédit.

Les neurones ne peuvent s'adapter les uns aux autres

Eviter l'overfitting des réseaux profonds par *dropout 3/3*

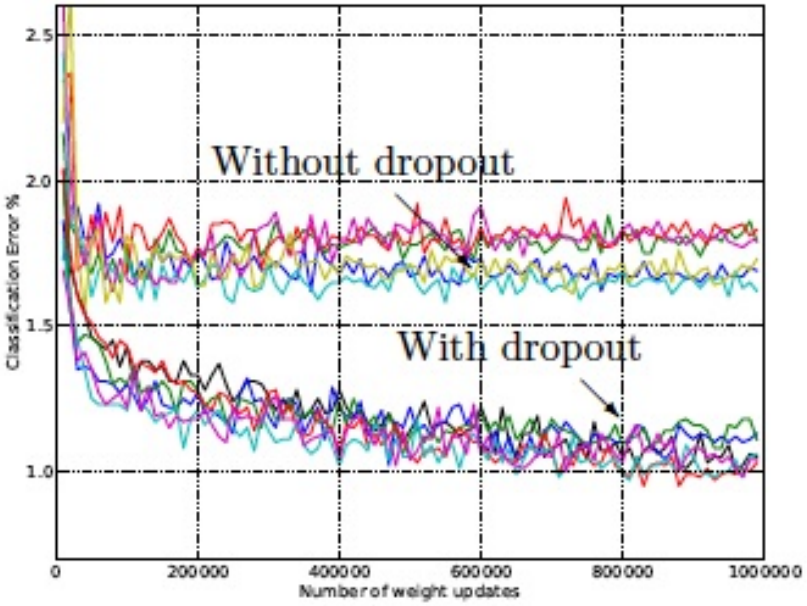


Figure 4: Test error for different architectures with and without dropout. The networks have 2 to 4 hidden layers each with 1024 to 2048 units.

Apprentissage des réseaux dits profonds

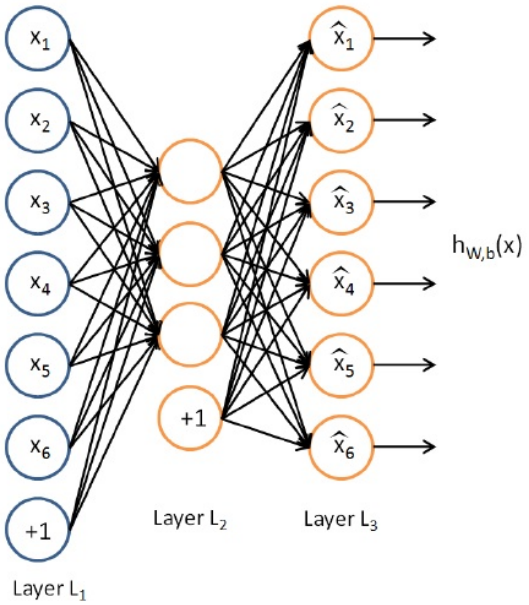
Les **autoencodeurs** sont de puissantes techniques de réduction de dimension (apprentissage non-supervisé) Ils sont également utilisés

dans certains cas pour initialiser les poids d'un réseau de neurones plus classique.

Autoencodeurs

Autoencodeurs

Un autoencodeur (aussi appelé *réseau diabolo*) est un réseau à une couche d'entrée, une ou plusieurs couches cachées et une couche de sortie. Ce type de réseau cherche à construire une représentation interne (la couche du milieu) en apprenant à prédire l'entrée à partir de celle-ci : $x \approx g(x)$.



Apprentissage d'autoencodeurs

- Un auto-encodeur s'apprend par rétropropagation du gradient
- Les autoencodeurs parcimonieux cherchent à limiter l'activité moyenne de chaque neurone de la couche intermédiaire (perte quadratique pénalisée)
- L'autoencodeur n'a d'autre intérêt que d'apprendre des représentations internes dans le cas de données complexes

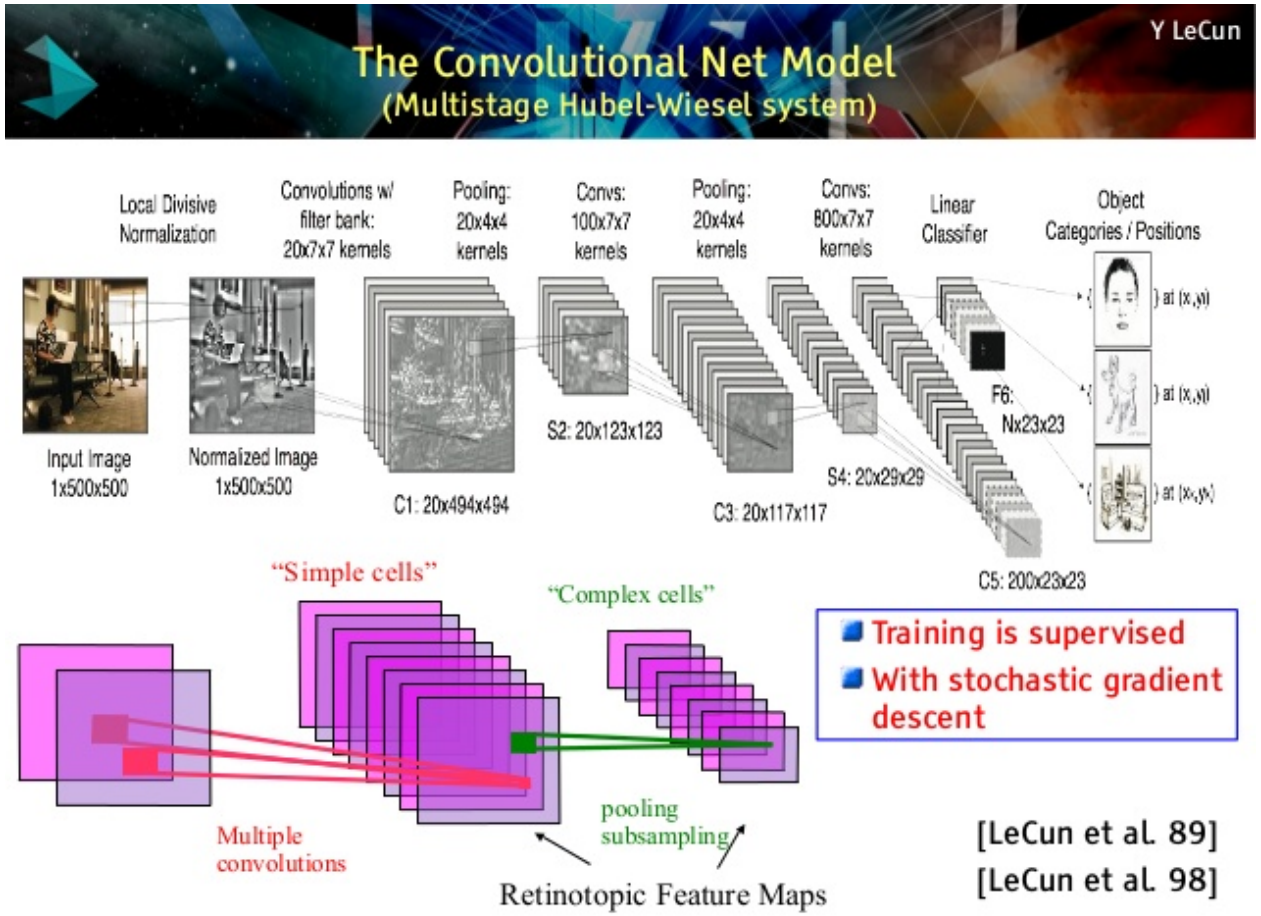
Autoencodeur et apprentissage d'un réseau "feed-forward" profond (Erhan et al.)

Pour chaque couche cachée en démarrant par la plus proche de l'entrée x, on définit ses poids en les extrayant de la première couche d'un autoencodeur appris sur :

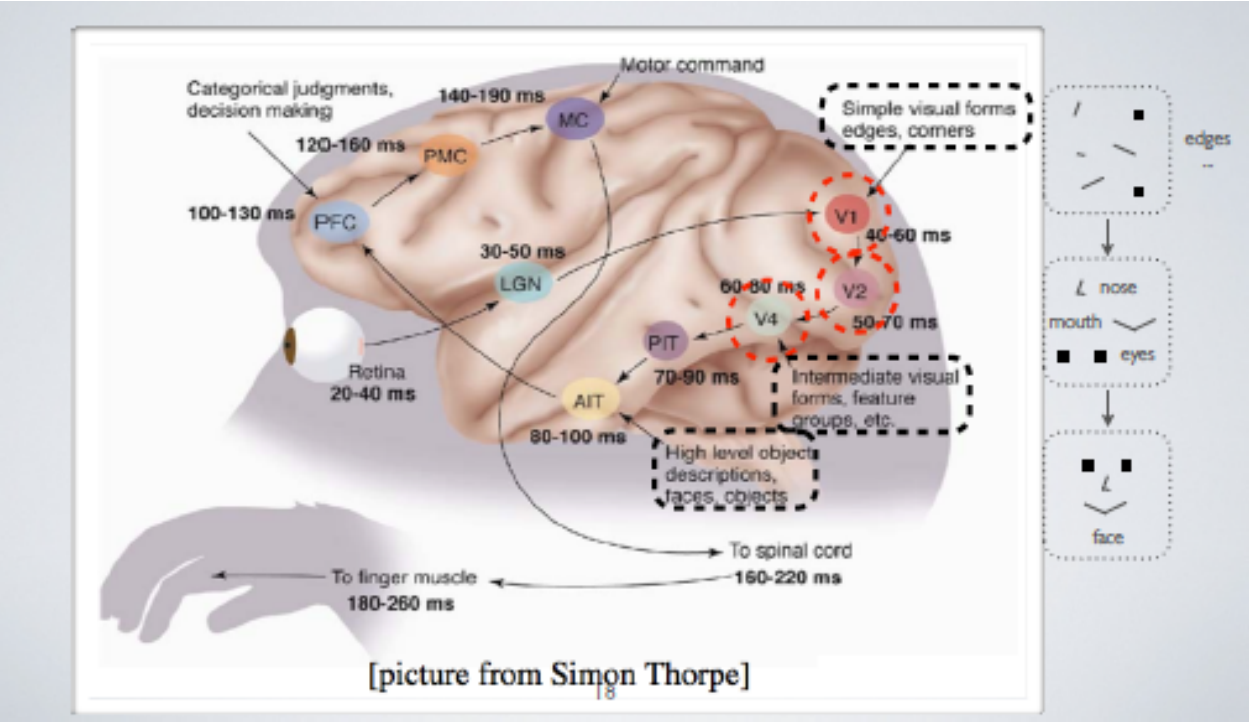
- Poids de la couche 2 : on apprend un autoencodeur $x \approx h(x)$. On initialise les poids de la couche 2 par ceux de la couche 2 de l'autoencodeur
- Poids de la couche 3 : on apprend un autoencodeur $f_1(x) \approx h(f_1(x))$. On initialise les poids de la couche 3 par ceux de la couche 2 de ce nouvel autoencodeur
- etc...
- Ensuite, on apprend de manière supervisée à partir de cette initialisation

Réseaux convolutionnels pour les images

Y. Le Cun.



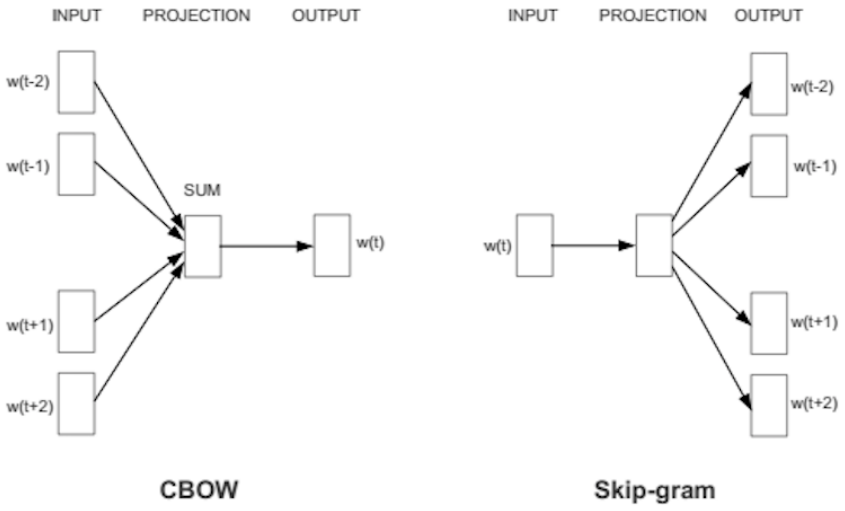
Cortex visuel



[picture from Simon Thorpe]

Skip-gram modele pour codage de mots

Mikolov et al. 2013.



NB : représentations continues des mots déjà proposées en 2002/2003.

Sommaire

1. Introduction
2. Classification binaire
3. Moyennes Locales
4. Ensemble Learning
5. Machines à Vecteurs Supports (SVM)
6. Réseaux de neurones
7. Références
8. Références

Références Ensemble Learning

- Y. Amit, D. Geman, and K. Wilder, Joint induction of shape features and tree classifiers, IEEE Trans. Pattern Anal. Mach. Intell., 19, 1300-1305, 1997.
- Breiman, L., Bagging predictors. Mach Learn (1996) 24 : 123.
- Y. Freund, R. Schapire, A decision-theoretic generalization of on-line learning and an application to boosting. In Computational Learning Theory, 1995.
- J. Friedman, T. Hastie and R. Tibshirani, Additive logistic regression : a statistical view of boosting. Ann. Statist. Vol. 28, No. 2 (2000), 337-407.
- Breiman, L., Random Forests. Mach. Learn. (2001), Vol. 45, No. 1, pp 5–32
- Tutorial : Ensemble Methods in Machine Learning. T.G. Dietterich, available at :
http ://web.engr.oregonstate.edu/ tgd/publications/mcs-ensembles.pdf

Références - Réseaux de Neurones

- Le cours de Hugo Larochelle (youtube)
- Notes de cours IT6266, Université de Montréal, Equipe de Yoshua Bengio.
- Learning Deep Architectures for AI, Yoshua Bengio, Foundations and Trends in Machine Learning, 2009
- Dropout : A simple way to prevent overfitting, Srivastava et al. JMLR 2014
- Pattern Recognition and Machine Learning, C. Bishop, Springer, 2006.
- `http://deeplearning.net/tutorial/` : pour tout document y compris implémentations...

Sommaire

1. Introduction
2. Classification binaire
3. Moyennes Locales
4. Ensemble Learning
5. Machines à Vecteurs Supports (SVM)
6. Réseaux de neurones
7. Références
8. Références

Références SVM

- BOSER, Bernhard E., Isabelle M. GUYON, and Vladimir N. VAPNIK, 1992. A training algorithm for optimal margin classifiers. In : COLT '92 : Proceedings of the Fifth Annual Workshop on Computational Learning Theory. New York, NY, USA : ACM Press, pp. 144-152.
- CORTES, Corinna, and Vladimir VAPNIK, 1995. Support-vector networks. Machine Learning, 20(3), 273–297.
- A tutorial review of RKHS methods in Machine Learning, Hoffman, Schölkopf, Smola, 2005 (https://www.researchgate.net/publication/228827159_A_Tutorial_Review_of_RKHS_Methods_in_Machine_Learning)

Références - Réseaux de Neurones

- Le cours de Hugo Larochelle (youtube)
- Notes de cours IT6266, Université de Montréal, Equipe de Yoshua Bengio.
- Learning Deep Architectures for AI, Yoshua Bengio, Foundations and Trends in Machine Learning, 2009
- Dropout : A simple way to prevent overfitting, Srivastava et al. JMLR 2014
- Pattern Recognition and Machine Learning, C. Bishop, Springer, 2006.
- `http://deeplearning.net/tutorial/` : pour tout document y compris implémentations...

Questions ?

Merci !